

Circuit Breaker Pattern in Microservices

Implementing Fault Tolerance in Microservice Architecture using Spring Boot, Netflix Eureka, and Resilience4j

Batuhan Orkun İNCE (212010020015)

Seyfullah Adıgüzel (212010020102)

ABSTRACT

This project researches and implements the **Circuit Breaker** design pattern for managing fault tolerance in a microservice architecture. The system is built using Spring Boot 3.2.5, Netflix Eureka for service discovery, and Resilience4j as the Circuit Breaker library. It consists of three distinct microservices containerized with Docker and orchestrated via Docker Compose, enabling isolated, reproducible, and single-command deployment. The primary goal is to simulate and prevent **cascading failures** — one of the most critical failure modes in distributed systems — and demonstrate controlled, graceful degradation as an alternative to total system collapse.

1. INTRODUCTION

In microservice architectures, services communicate with each other over the network. This distributed nature introduces a fundamental vulnerability: a delay or crash in a single service can block all dependent services, propagating failure across the entire system in a chain reaction known as a **cascading failure**.

Without a protection mechanism, a single unresponsive service can:

- Lock all available threads in dependent services
- Cause memory exhaustion under increasing load
- Bring down the entire system, not just one component
- Leave users with no response — not even a graceful error message

The **Circuit Breaker** design pattern was implemented in this project specifically to intercept and contain this class of failures before they propagate. Rather than allowing a slow or failed downstream service to hold up the entire call chain, the Circuit Breaker detects the failure and immediately redirects execution to a predefined fallback method, keeping the rest of the system alive and responsive.

2. TECHNOLOGY BACKGROUND

2.1 Spring Boot

Spring Boot is a framework for building standalone, production-ready Spring applications. It provides auto-configuration and embedded servers (Tomcat), enabling rapid microservice development without

complex XML configuration. All three services in this project are Spring Boot applications compiled to self-contained JAR files. The versions used are Spring Boot 3.2.5 and Java 17.

2.2 Netflix Eureka

Netflix Eureka is a **Service Discovery** system. In a microservice environment, services need to know each other's network addresses without hardcoding them. Eureka solves this by acting as a central registry: each service registers itself with Eureka on startup, advertising its name and address. Other services query Eureka by service name (e.g., `PRODUCT-SERVICE`) to resolve the actual host and port at runtime. This enables dynamic, location-transparent communication — services can move, scale, or restart without breaking the call chain.

2.3 Resilience4j

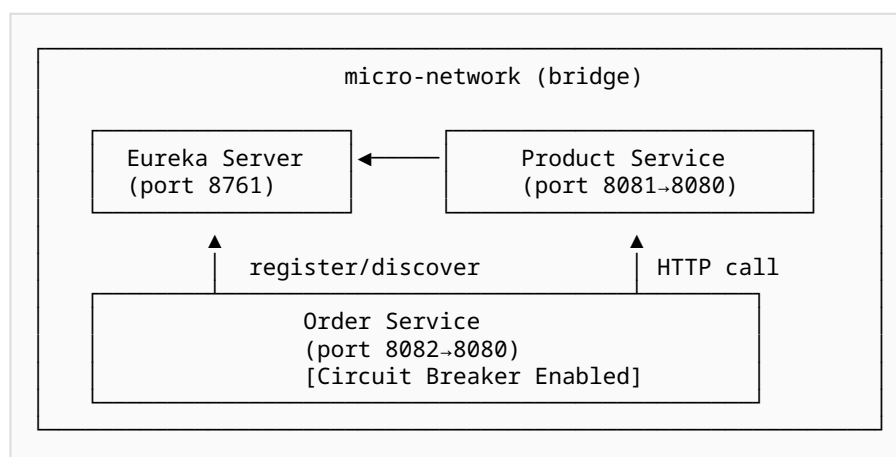
Resilience4j is a lightweight fault-tolerance library designed for Java 8+ and Spring Boot. It implements the Circuit Breaker pattern, monitoring call outcomes and automatically switching to fallback logic when failure thresholds are exceeded. Key advantages over the now-deprecated Netflix Hystrix include no external dependency on a thread pool management layer, fine-grained annotation-driven configuration per service instance, native integration with Spring Boot Actuator, and composability with `TimeLimiter`.

2.4 Docker & Docker Compose

Docker is a containerization platform that packages applications together with all their dependencies into isolated, portable units called containers. Docker Compose is a tool for defining and running multi-container Docker applications using a single YAML configuration file. In this project, each microservice is compiled into its own Docker image and all three are orchestrated together via `docker-compose.yml`, sharing a single internal bridge network (`micro-network`).

3. SYSTEM ARCHITECTURE

The system is composed of exactly **three microservices**, each with a distinct role inside the internal bridge network.



3.1 Eureka Server (Service Registry)

The central registration hub to which all other services register themselves on startup. It acts as the address book of the system — services query it to discover the network location of their dependencies. Without this component, services cannot find each other dynamically at runtime. The exposed port is 8761.

3.2 Product Service

The core, independent microservice that handles product data requests. It exposes a single REST endpoint (GET /products) that returns a static list of product names. For testing and demonstration purposes, it is designed to simulate failure scenarios — allowing the Circuit Breaker behavior in Order Service to be triggered and observed. The exposed port is 8081 (mapped to internal 8080).

3.3 Order Service (Circuit Breaker Enabled)

The service that depends on Product Service. When a request arrives at GET /orders, it calls http://PRODUCT-SERVICE/products via the Eureka-resolved address. If Product Service fails to respond within the configured timeout (1 second) or throws an error, the **Resilience4j Circuit Breaker** activates and routes execution to the fallback method, returning a controlled error message to the caller rather than freezing. The exposed port is 8082 (mapped to internal 8080).

4. IMPLEMENTATION DETAILS

4.1 Order Service — Circuit Breaker Configuration

The Circuit Breaker is configured in `order-service/src/main/resources/application.yml`:

```
resilience4j:
  circuitbreaker:
    instances:
      productService:
        registerHealthIndicator: true
        slidingWindowSize: 10
        permittedNumberOfCallsInHalfOpenState: 3
        slidingWindowType: COUNT_BASED
        minimumNumberOfCalls: 5
        waitDurationInOpenState: 10s
        failureRateThreshold: 50
  timelimiter:
    instances:
      productService:
        timeoutDuration: 1s
        cancelRunningFuture: true
```

Table 1: Circuit Breaker Configuration parameters breakdown.

Parameter	Value	Meaning
slidingWindowSize	10	Evaluates the last 10 calls to determine health
minimumNumberOfCalls	5	Needs at least 5 calls before making a decision
failureRateThreshold	50	Opens circuit when $\geq 50\%$ of calls fail
waitDurationInOpenState	10s	Stays Open for 10 seconds before trying Half-Open
permittedNumberOfCallsInHalfOpenState	3	Allows 3 probe calls in Half-Open state
timeoutDuration	1s	Any call taking longer than 1 second counts as a failure

4.2 Order Service — Controller Implementation

```

@GetMapping
@CircuitBreaker(name = "productService", fallbackMethod = "fallbackMethod")
@TimeLimiter(name = "productService")
public CompletableFuture<String> placeOrder() {
    logger.info("Order request received. Connecting to PRODUCT-SERVICE...");
    return CompletableFuture.supplyAsync(() -> {
        List<?> productList = restTemplate
            .getForObject("http://PRODUCT-SERVICE/products", List.class);
        logger.info("Response from product service received. Order complete.");
        return "Order successful! Products: " + productList.toString();
    });
}

public CompletableFuture<String> fallbackMethod(Exception e) {
    logger.error("ALERT: Product service unreachable! Circuit Breaker activated.
Error: {}",
        e.getMessage());
    return CompletableFuture.supplyAsync(() ->
        "Üzgünüz, Ürün Servisinde bir arıza var (Circuit Breaker devrede!) Sistem
çökmedi."
    );
}

```

4.3 Product Service — Controller Implementation

```

@GetMapping
public List<String> getProducts() {
    logger.info("Product list request received. Sending products...");
    return Arrays.asList("Laptop", "Akıllı Telefon", "Tablet", "Kulaklık");
}

```

4.4 Docker Compose — Orchestration Configuration

```
services:
  eureka-server:
    image: eureka-server:latest
    build: ./eureka-server
    ports:
      - "8761:8761"
    networks:
      - micro-network

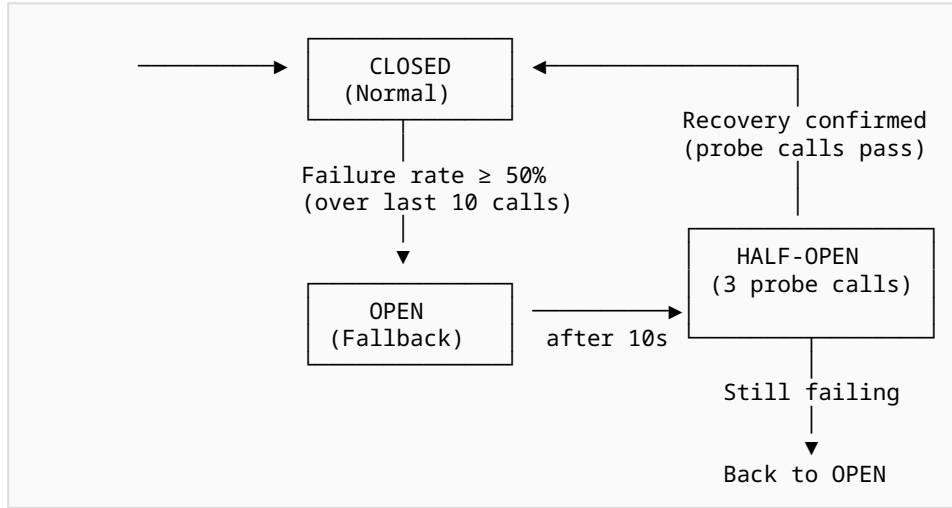
  product-service:
    image: product-service:latest
    build: ./product-service
    environment:
      - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka/
    ports:
      - "8081:8080"
    depends_on:
      - eureka-server
    networks:
      - micro-network

  order-service:
    image: order-service:latest
    build: ./order-service
    environment:
      - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka/
    ports:
      - "8082:8080"
    depends_on:
      - eureka-server
      - product-service
    networks:
      - micro-network

networks:
  micro-network:
    driver: bridge
```

5. CIRCUIT BREAKER STATES

The Resilience4j Circuit Breaker operates as a finite state machine with three major states:



5.1 Closed State (Normal Operation)

All calls pass through to Product Service normally. The Circuit Breaker monitors the failure rate silently in the background using a sliding window of the last 10 calls. No interventions are performed while the system is healthy.

5.2 Open State (Fault Isolation)

When the failure rate surpasses the 50% threshold over the minimum required calls, the circuit opens. All subsequent calls are immediately redirected to the fallback method without attempting to reach Product Service. No threads are blocked, saving system resources. The circuit stays Open for exactly 10 seconds.

5.3 Half-Open State (Recovery Probe)

After the 10-second wait duration, the circuit transitions to Half-Open and allows exactly 3 probe requests through to Product Service. If these calls succeed, the circuit closes and normal operation resumes. If they fail, the circuit returns to the Open state for another wait cycle.

6. FAILURE SCENARIOS AND RESILIENCE ANALYSIS

6.1 Failure Scenario 1 — Downstream Service Failure

When Product Service crashes, Order Service waits for a response. Without a Circuit Breaker, all available threads become locked waiting for the unresponsive downstream service. As incoming load increases, Order Service exhausts its thread pool and crashes. With Resilience4j, when Product Service goes down, calls start timing out. After the failure rate exceeds 50%, the Circuit Breaker transitions to the Open state, immediately invoking `fallbackMethod` and returning a graceful response: *"Üzgünüz, Ürün Servisinde bir arıza var (Circuit Breaker devrede!) Sistem çökmedi."*

6.2 Failure Scenario 2 — Service Registry Failure

If Eureka Server crashes, microservices can no longer look up each other's addresses dynamically. However, Eureka clients cache the registry data locally in memory. Even if Eureka crashes, the microservices retain the last-known IP addresses and continue communicating via these cached entries for a short grace period. This behavior is built into the Netflix Eureka client library and requires no additional manual intervention.

7. DEPLOYMENT AND VERIFICATION

The system is deployed using the command `docker-compose up --build` from the root folder. Verification is done via the Eureka Dashboard (`http://localhost:8761`), Product Endpoint (`http://localhost:8081/products`), and Order Endpoint (`http://localhost:8082/orders`). Testing the circuit breaker involves invoking `docker stop product-service` and confirming that subsequent calls return the fallback text string gracefully.

8. KEY RESULTS AND CONCLUSION

This project successfully implemented the Circuit Breaker design pattern in a microservice architecture. The cascading failure scenario was completely prevented. When Product Service was shut down, Order Service remained functional, returning a human-readable fallback message instead of freezing. Docker integration allowed full portability, and Eureka client caching provided an additional layer of architectural durability. The results prove that distributed systems can fail gracefully under high load and partial component unavailability when managed by modern fault-tolerance primitives.

APPENDIX: TECHNOLOGY STACK SUMMARY

Component	Technology	Version
Framework	Spring Boot	3.2.5
Language	Java	17
Service Discovery	Netflix Eureka	Spring Cloud 2023.0.1
Circuit Breaker	Resilience4j	Spring Cloud 2023.0.1
Containerization & Orchestration	Docker / Docker Compose	Latest